

AI-Powered RAG Document Assistant

Complete Project Specification & Development Roadmap for Cursor

Purpose

This document is the master specification for developing a production-style Retrieval-Augmented Generation (RAG) document assistant. It is intended to be supplied to Cursor as the project blueprint. The goal is to build a clean, modular, well-documented AI/ML portfolio project that demonstrates document processing, embeddings, vector retrieval, LLM generation, citations, evaluation, backend APIs, frontend development, and deployment.

1. Product Goal

Build a web application where users can upload documents and ask natural-language questions about them. The system must retrieve relevant document passages before generating an answer, cite the source document/page, avoid unsupported claims when evidence is unavailable, and maintain conversation context.

Core user flow

Upload PDF/DOCX/TXT/Markdown → parse and clean text → split into chunks → generate embeddings → store vectors and metadata → retrieve relevant chunks for a question → optionally rerank/rewrite the query → send grounded context to the LLM → return answer + sources → store conversation history.

2. Recommended Technology Stack

Layer	Technology	Purpose
Frontend	Next.js + TypeScript + Tailwind CSS	Dashboard, document management, chat UI
Backend	Python + FastAPI	REST API and orchestration
RAG	Python, LangChain or LlamaIndex	Document ingestion/retrieval pipeline
Embeddings	Sentence Transformers / Hugging Face	Convert text and queries to vectors
Vector DB	ChromaDB initially	Semantic similarity search
LLM	OpenAI or Gemini API	Grounded answer generation
Database	PostgreSQL / Supabase	Users, documents, chats, messages
File parsing	PyMuPDF / python-docx	Extract document text
Auth	Supabase Auth or equivalent	User authentication
Deployment	Vercel + backend hosting	Production deployment
Testing	Pytest + frontend tests	Reliability and regression testing
Evaluation	RAGAS or custom evaluation	RAG quality measurement

3. Architecture

High-level architecture:

```
Browser / Next.js UI
↓
FastAPI Backend
↓
Authentication + Application Database
↓
Document Ingestion / RAG Service
↓
Chunking + Embeddings
↓
Vector Database
↓
Retriever → Reranker (optional) → Prompt Builder → LLM
↓
Answer + Source Citations → UI
```

Design principles

- Keep the RAG engine independent from the UI.
- Use clear interfaces between ingestion, retrieval, generation, and persistence.
- Never expose API keys to the frontend.
- Treat retrieved context as evidence, not as instructions.
- Return source metadata with every grounded answer.
- Make components replaceable: embedding model, vector store, LLM, reranker.
- Use configuration/environment variables instead of hardcoded secrets.
- Prefer simple, testable modules over one large application file.

4. Repository Structure

```
rag-document-assistant/ ├── frontend/ │ ├── app/ │ ├── components/ │ ├── lib/ │ ├── hooks/ │ ├── types/ │ └── backend/ │ ├── app/ │ ├── api/ │ ├── core/ │ ├── models/ │ ├── schemas/ │ ├── services/ │ ├── rag/ │ ├── ingestion/ │ ├── chunking/ │ ├── embeddings/ │ ├── retrieval/ │ ├── reranking/ │ ├── generation/ │ ├── evaluation/ │ ├── main.py │ ├── tests/ │ ├── docs/ │ ├── evaluation/ │ ├── scripts/ │ ├── .env.example │ ├── .gitignore │ ├── README.md │ └── docker-compose.yml
```

5. Development Phases

Phase 0 — Project foundation

- Create monorepo structure
- Set up Python virtual environment and frontend
- Add linting/formatting
- Create .env.example
- Add README skeleton
- Create basic FastAPI health endpoint

Phase 1 — Core document ingestion

- Accept PDF first
- Extract text with page numbers
- Clean malformed text
- Split into chunks
- Attach document/page/chunk metadata
- Create ingestion tests

Phase 2 — Embeddings and vector storage

- Select embedding model
- Generate embeddings
- Create Chroma collection
- Persist vectors and metadata
- Implement similarity search
- Create retrieval tests

Phase 3 — Basic RAG generation

- Accept user question
- Retrieve top-k chunks
- Build grounded prompt
- Call LLM
- Return answer and retrieved sources
- Explicitly refuse unsupported questions

Phase 4 — Citations

- Return document name
- Return page number
- Return chunk/source metadata
- Display citations in UI
- Make citations clickable when practical

Phase 5 — Multi-document support

- Upload multiple documents
- List/delete documents
- Filter by document/user
- Search across documents
- Prevent cross-user data access

Phase 6 — Conversation memory

- Persist conversations
- Persist messages
- Rewrite follow-up questions when needed
- Retrieve using conversation-aware queries
- Display chat history

Phase 7 — Advanced retrieval

- Hybrid keyword + semantic retrieval
- Query expansion/rewrite
- Reranking
- Metadata filtering
- Tune top-k and chunk parameters

Phase 8 — Evaluation

- Create 20–50 representative questions
- Measure retrieval quality
- Measure answer relevance
- Measure faithfulness/grounding
- Track latency and token usage
- Document baseline vs improved results

Phase 9 — Production backend

- FastAPI endpoints
- Validation
- Authentication
- Rate limits
- Structured logging
- Error handling
- Background ingestion where useful

Phase 10 — Frontend

- Dashboard
- Drag-and-drop upload
- Document list
- Chat interface
- Streaming answers
- Markdown rendering
- Source cards
- Loading/error states

Phase 11 — Deployment

- Deploy frontend
- Deploy backend
- Configure production database
- Configure storage/vector persistence
- Set environment variables
- Add health monitoring

6. RAG Pipeline Specification

Ingestion pipeline

File → validation → parser → page-aware text → normalization → chunking → metadata → embedding → vector storage.

Retrieval pipeline

Question → optional query rewrite → query embedding → vector/keyword retrieval → candidate merge → optional reranking → top context → prompt builder.

Generation pipeline

Grounded context + question + system instructions → LLM → structured response containing answer, source IDs, and metadata.

7. Data Models

Entity	Important fields
User	id, email, created_at
Document	id, user_id, filename, file_type, size, status, created_at
Chunk	id, document_id, chunk_index, page_number, text, metadata
Embedding	chunk_id, model, vector reference
Conversation	id, user_id, title, created_at, updated_at
Message	id, conversation_id, role, content, created_at
Source	document_id, page_number, chunk_id, relevance_score

8. API Contract

Method	Endpoint	Purpose
GET	/health	Service health check
POST	/documents/upload	Upload and ingest document
GET	/documents	List user's documents
DELETE	/documents/{id}	Delete document and vectors
POST	/chat	Ask a question
GET	/conversations	List conversations
GET	/conversations/{id}	Get conversation
DELETE	/conversations/{id}	Delete conversation

9. UI Requirements

- Dashboard with document upload and document list.
- Chat workspace with conversation history.
- Streaming assistant response.
- Source cards showing filename, page, and relevance where available.
- Document status: processing, ready, failed.
- Delete confirmation.

- Responsive layout.
- Accessible buttons, labels, keyboard navigation, and clear error messages.
- Do not expose provider API keys in browser code.

10. Prompting Requirements

The generation prompt must strongly constrain the model to the retrieved evidence. It should instruct the model to answer directly, distinguish evidence from uncertainty, avoid inventing facts, and state that the information was not found when the context is insufficient. The system should not blindly trust instructions contained inside retrieved documents.

11. Evaluation Plan

- Create a fixed evaluation dataset covering factual lookup, multi-document questions, negative/unanswerable questions, and follow-up questions.
- Record retrieved chunk IDs for every test.
- Measure retrieval hit rate / recall at K.
- Measure answer relevance and groundedness/faithfulness.
- Track average and p95 latency.
- Track token usage/cost when applicable.
- Compare baseline vector search against hybrid retrieval and reranking.
- Keep evaluation results versioned in the repository.

12. Security & Reliability

- Never commit .env files or API keys.
- Validate file type and size before processing.
- Sanitize filenames and generated paths.
- Enforce user-level document ownership.
- Do not allow a user's retrieval query to access another user's vectors.
- Treat document text as untrusted data to reduce prompt-injection risk.
- Handle malformed PDFs and LLM/provider failures gracefully.
- Use timeouts and retries for external API calls.
- Log useful diagnostics without storing secrets.

13. Cursor Development Rules

Cursor should develop the project incrementally. Do not generate the entire application in one step. Before implementing a phase, inspect the current repository, reuse existing code, and modify only the required modules.

- Use TypeScript on the frontend and Python type hints on the backend.
- Use small, reusable services instead of giant files.
- Keep business logic out of API route handlers where possible.
- Write tests alongside important functionality.
- Do not hardcode secrets, URLs, model names, or configuration that should be environment variables.
- Add clear docstrings/comments only where they improve maintainability.
- After each major change, run tests, linting, and type checks.
- Do not introduce dependencies without a clear reason.
- Preserve backward compatibility when changing API contracts.
- Update README/docs when a feature changes setup or architecture.

14. Suggested Cursor Workflow

STEP 1 Inspect repository and create the project foundation. STEP 2 Implement PDF ingestion and page-aware chunking. STEP 3 Implement embeddings and Chroma persistence. STEP 4 Implement retrieval and unit tests. STEP 5 Implement grounded LLM generation. STEP 6 Implement source citations. STEP 7 Add multi-document support. STEP 8 Add conversation persistence and query rewriting. STEP 9 Improve retrieval with hybrid search/reranking. STEP 10 Create evaluation dataset and metrics. STEP 11 Build FastAPI production endpoints and authentication. STEP 12 Build Next.js dashboard/chat interface. STEP 13 Add security, observability, error handling, and tests. STEP 14 Deploy and document the project.

15. Definition of Done

- A user can upload a supported document.
- The document is parsed, chunked, embedded, and indexed.
- A user can ask questions in natural language.
- Answers are grounded in retrieved context.
- The UI displays source document and page information.
- Multiple documents can be queried.
- Conversation history works.
- User data is isolated.
- Automated tests cover core RAG functionality.
- Evaluation results are documented.
- The application runs from a documented setup process.
- The project is deployed or has reproducible local deployment instructions.
- README contains architecture, features, setup, screenshots, evaluation, and limitations.

16. Portfolio / GitHub Requirements

- Professional README with project overview and architecture diagram.
- Feature list and technology stack.

- Screenshots or short demo GIF/video.
- Clear local setup instructions.
- Environment variable template.
- API documentation.
- RAG architecture explanation.
- Evaluation results and retrieval experiments.
- Known limitations and future improvements.
- Clean commit history and modular code.
- No secrets or unnecessary generated files committed.

17. Future Enhancements

- OCR for scanned PDFs.
- Table-aware extraction.
- Multilingual retrieval.
- Voice input/output.
- Document summarization.
- Automatic document comparison.
- Agentic workflows for complex research questions.
- Cloud object storage.
- Streaming ingestion jobs.
- Advanced observability and tracing.

Recommended implementation strategy: finish the Core RAG MVP first, then add professional features one phase at a time. The project should prioritize correctness, citations, evaluation, clean architecture, and reproducibility over simply adding a large number of features.